

Big Data CheatSheet

HDFS

HDFS (Hadoop Distributed File System) is a distributed, scalable, and fault-tolerant file system designed to store large files across a cluster of commodity hardware.

- **Blocks:** Files in HDFS are broken into large blocks (typically 128MB or 256MB). Each block is stored independently.
- **Replication:** Each block is replicated across multiple nodes (default is 3) to ensure fault tolerance. The replicas are stored on different racks to avoid data loss due to rack failure.
- **NameNode:** The master server that manages the file system namespace and regulates access to files by clients. It stores the metadata of the files (e.g., which blocks make up a file, and on which DataNodes those blocks are stored).
- **DataNodes:** The slave nodes that store the actual data blocks. They are responsible for serving read and write requests from the clients.
- **Write-once-read-many:** HDFS is optimized for large, streaming reads of data. Files are typically written once and read many times. Appends are possible, but in-place modifications are not.

MapReduce Theory

MapReduce is a programming model for processing large data sets with a parallel, distributed algorithm on a cluster.

- **Map Phase:** The input data is split into independent chunks which are processed by the map tasks in a completely parallel manner. The map function takes a key-value pair and emits a set of intermediate key-value pairs.
- **Shuffle and Sort:** The framework shuffles and sorts the intermediate key-value pairs from the mappers. All pairs with the same key are grouped together.

- **Reduce Phase:** The reduce function is called for each unique key and its list of associated values. It processes the list of values and emits a new set of key-value pairs, which are the final output.
- **Data Locality:** Hadoop moves the computation (code) to the data, not the other way around. This minimizes network traffic and is a key principle for performance.

Hadoop MapReduce Implementation

Hadoop MapReduce Job Execution

- **InputSplit:** An 'InputSplit' is a chunk of the input that is processed by a single map task. By default, the 'InputFormat' creates splits that are the size of an HDFS block.
- **Number of Mappers:** The number of map tasks is determined by the number of 'InputSplit's. If you have a 1GB file stored in HDFS with a block size of 128MB, you will have 8 blocks and therefore 8 'InputSplit's, resulting in 8 map tasks.
- **Number of Reducers:** The number of reduce tasks is configured by the developer in the driver code using `job.setNumReduceTasks(N)`. If N is 0, no reducers will be run (map-only job).
- **Output Files:** The number of output files is equal to the number of reducers. Each reducer writes its output to a separate file in the output directory. For map-only jobs, each mapper writes its output to a separate file.

Function Calls

Function Call Frequency

- **setup():** Called **once** per task (both for mappers and reducers) before any other method. Used for initialization.
- **map():** Called **once for each key-value pair** in the 'InputSplit' assigned to the mapper.

- **reduce():** Called **once for each unique key** and its associated list of values.
- **cleanup():** Called **once** per task (both for mappers and reducers) after all other methods have been executed. Used for finalization.

Combiner

Combiner (Mini-Reducer)

- A Combiner is an optional optimization that performs a local reduce on the map-side. It takes the output of a map task and aggregates it before sending it over the network to the reducers.
- This reduces the amount of data transferred over the network.
- The Combiner class is usually the same as the Reducer class, but it's not a requirement. `job.setCombinerClass(MyReducer.class)`
- **Execution is not guaranteed.** Hadoop may or may not execute the combiner. Therefore, the logic of a MapReduce job should not depend on the combiner's execution.
- For a combiner to be used, the reduce function must be associative and commutative.

Job Configuration (I)

Job Configuration (Driver): The 'Job' object is used in the driver to configure the MapReduce job.

- **setJarByClass(class):** Sets the Jar by finding where a given class came from.
- **setJobName(name):** Sets the name of the job.
- **setMapperClass(class):** Sets the 'Mapper' class.
- **setReducerClass(class):** Sets the 'Reducer' class.

Job Configuration (II)

- **setCombinerClass(class)**: Sets the 'Combiner' class.
- **setNumReduceTasks(num)**: Sets the number of reduce tasks.
- **setInputFormatClass(class)**: Sets the 'InputFormat' for the job.
- **setOutputFormatClass(class)**: Sets the 'OutputFormat' for the job.
- **setMapOutputKeyClass(class)**: Sets the key class for the map output.
- **setMapOutputValueClass(class)**: Sets the value class for the map output.
- **setOutputKeyClass(class)**: Sets the key class for the final output.
- **setOutputValueClass(class)**: Sets the value class for the final output.

Hadoop Data Types

Hadoop Data Types: Hadoop uses its own data types for serialization.

- **Writable**: Interface for objects that can be serialized. Requires implementing `write(DataOutput)` and `readFields(DataInput)`.
- **WritableComparable**: Interface for objects that are both writable and comparable. Keys must implement this. Requires implementing `compareTo(Object)`.
- **Common Types**:
 - **Text**: for strings.
 - **IntWritable**: for integers.
 - **LongWritable**: for long integers.
 - **FloatWritable**: for float values.
 - **DoubleWritable**: for double values.
 - **NullWritable**: a singleton for null values.

Input/Output Formats

Input/Output Formats

- **TextInputFormat**: Default. Key is the byte offset of the line (LongWritable), Value is the line content (Text).
- **KeyValueTextInputFormat**: Splits lines based on a separator (tab by default). Key is the text before the separator, Value is the text after.
- **SequenceFileInputFormat**: An input format for binary sequence files.
- **TextOutputFormat**: Default. Writes key-value pairs on a line, separated by a tab.

Context, Config & Counters

Context, Configuration, and Counters

1. Setting Parameters (in Driver)

- **conf.set()**
 - **Prototype**: `void set(String name, String value)`
 - **Description**: Sets a custom parameter in the job's 'Configuration' object. This is done in the Driver.
 - **Example**: `conf.set("my.parameter", "my.value");`

2. Accessing Parameters (in Mapper/Reducer)

- **context.getConfiguration().get()**
 - **Prototype**: `String get(String name)`
 - **Description**: Retrieves a parameter value set in the driver.
 - **Example**:
`String p = context.getConfiguration().get("my.parameter");`

3. Defining Counters (in Driver)

- Counters are defined as a Java `enum`. This provides a group name (the enum name) and counter names (the enum values). This is typically defined within the Driver class.
- **Example**: `public static enum MY_COUNTERS { NUM_RECORDS, MALFORMED_RECORDS }`

4. Using Counters (in Mapper/Reducer)

- **context.getCounter(...).increment()**
 - **Prototype**: `void increment(long value)`
 - **Description**: Retrieves a 'Counter' object and increments it by a given value.
 - **Example**:
`context.getCounter(MY_COUNTERS.NUM_RECORDS).increment(1);`

5. Emitting Output (in Mapper/Reducer)

- **context.write()**
 - **Prototype**: `void write(KEYOUT key, VALUEOUT value)`
 - **Description**: Emits a key-value pair from the task.
 - **Example**: `context.write(word, one);`

Summarization Patterns (I)

Summarization Patterns are used when you need to aggregate data.

- **Counting/Sum**: The classic WordCount is an example. Mappers emit a key and a number (e.g., 1), and reducers sum the numbers for each key.

Summarization Patterns (II)

- **Min/Max:** Mappers emit a key and the numeric value. Reducers iterate through the values for a key and find the minimum or maximum. The in-mapper combining pattern is very effective here.
- **Average:** Requires keeping track of both a sum and a count.
 - **Mapper:** Emits (`key`, (`sum`, `count`)).
 - **Combiner:** Combines pairs locally, summing both sums and counts. Emits (`key`, (`partial_sum`, `partial_count`)).
 - **Reducer:** Receives (`key`, [(`s1`, `c1`), (`s2`, `c2`), ...]) and computes the final average as (`total_sum` / `total_count`).

Filtering Pattern

Filtering Pattern: Used to select a subset of the data.

- This is a **map-only** job.
- The mapper function inspects each record and decides whether to keep it or discard it.
- If the record meets the criteria, the mapper writes it to the output. Otherwise, it does nothing.
- The number of reducers is set to 0.
`job.setNumReduceTasks(0);`

Sorting Pattern (Secondary Sort)

Sorting Pattern: MapReduce automatically sorts by key between the map and reduce phases. For more complex sorting, a **secondary sort** is needed.

- To sort by value (or part of the key), you create a composite key, e.g., (`primary_key`, `secondary_key`).

Sorting Pattern (Secondary Sort)

- **Custom Partitioner:** Ensures all keys with the same `primary_key` go to the same reducer. It partitions based on only the `primary_key` part of the composite key.
- **Custom Grouping Comparator:** Tells the reducer to group records based on only the `primary_key`. This way, the reducer gets all values for that `primary_key` in one call, with the values sorted by the `secondary_key`.

Reduce-Side Join

Reduce-Side Join: A common way to join two datasets.

- **Mapper:** Processes both datasets. For each record, it emits the join key as the intermediate key. The value is a pair containing a "tag" to identify the source dataset and the original value. E.g., (`join_key`, (`"data1"`, `value1`)) and (`join_key`, (`"data2"`, `value2`)).
- **Reducer:** Receives all values for a given join key. It inspects the tags to separate the values from each dataset and performs the join logic (e.g., a Cartesian product). This can be slow if some keys have many values.

Map-Side Join

Map-Side Join (Replicated Join): Used when one dataset is small enough to fit into the memory of each mapper.

- The small dataset is loaded into each mapper's memory in the `setup()` method. This is often done using the `DistributedCache`.
- **Mapper:** Processes the large dataset. For each record, it looks up the corresponding data from the small dataset in its in-memory map and performs the join. The result is emitted directly.
- This is a **map-only job** and is much more efficient than a reduce-side join as it avoids network shuffling.

Custom Partitioner

Custom Partitioner: Controls which reducer receives which intermediate keys.

- By default, keys are partitioned by `hash(key) % num_reducers`.
- A custom partitioner is a class that extends `Partitioner` and implements the `getPartition(...)` method.
- **getPartition(key, value, numPartitions):**
 - **Prototype:** `int getPartition(K key, V value, int numPartitions)`
 - **Description:** Returns the partition number (i.e., reducer index) for the given key-value pair.
 - **Usage:** Essential for secondary sort to ensure all parts of a composite key go to the same reducer.
 - **Set in Driver:**
`job.setPartitionerClass(MyPartitioner.class);`

RDD Theory

RDD Theory

- **RDD (Resilient Distributed Dataset):** The fundamental data structure of Spark. It is an immutable, distributed collection of objects.
- **Lazy Execution:** Transformations on RDDs are not computed immediately. Spark builds up a **DAG (Directed Acyclic Graph)** of operations. The computations are only executed when an action is called.
- **Transformations:** Create a new RDD from an existing one (e.g., `map`, `filter`). They are lazy.
- **Actions:** Trigger the execution of the DAG and return a value to the driver program or write data to an external storage system (e.g., `count`, `collect`).

SparkContext

SparkContext

- **Creation:**

- from pyspark import SparkConf, SparkContext
- conf = SparkConf().setAppName(name)
- sc = SparkContext(conf=conf)

- **parallelize(list, numSlices):** Creates an RDD from a Python list.
- **textFile(path, minPartitions):** Reads a text file and returns it as an RDD of strings.
- **stop():** Shuts down the SparkContext.

RDD Transformations (I)

RDD Transformations (Basic)

- **map(f):** map(f: function) -> RDD
 - Returns a new RDD by applying a function to each element.
 - rdd.map(lambda x: x + 1)
- **flatMap(f):** flatMap(f: function) -> RDD
 - Similar to map, but each input item can be mapped to 0 or more output items.
 - rdd.flatMap(lambda s: s.split(' '))
- **filter(f):** filter(f: function) -> RDD
 - Returns a new RDD containing only the elements that satisfy a function.
 - rdd.filter(lambda x: x > 10)
- **distinct():** distinct() -> RDD
 - Returns a new RDD containing the distinct elements.
 - rdd.distinct()

- **sample(withReplacement, fraction, seed):** sample(...) -> RDD

- Return a sampled subset of this RDD.
- rdd.sample(False, 0.5)

RDD Transformations (II)

RDD Transformations (Pair RDDs)

- **reduceByKey(func):** reduceByKey(func) -> RDD
 - Merge the values for each key using an associative and commutative reduce function.
 - rdd.reduceByKey(lambda a, b: a + b)
- **groupByKey():** groupByKey() -> RDD
 - Group the values for each key in an RDD into a single sequence.
 - rdd.groupByKey()
- **sortByKey(ascending):** sortByKey(asc) -> RDD
 - Sorts this RDD, which is assumed to consist of (key, value) pairs.
 - rdd.sortByKey()
- **join(other):** join(other) -> RDD
 - Return an RDD containing all pairs of elements with matching keys in self and other.
 - rdd1.join(rdd2)
- **leftOuterJoin(other):** leftOuterJoin(other) -> RDD
 - Perform a left outer join of self and other.
 - rdd1.leftOuterJoin(rdd2)

- **cogroup(other):** cogroup(other) -> RDD

- For each key k in self or other, return a resulting RDD that contains a tuple with the list of values for that key in self as well as other.

- rdd1.cogroup(rdd2)

- **rightOuterJoin(other):** rightOuterJoin(other) -> RDD

- Perform a right outer join of self and other.
- rdd1.rightOuterJoin(rdd2)

RDD Actions (I)

RDD Actions

- **collect():** collect() -> list
 - Return a list that contains all of the elements in this RDD.
 - rdd.collect()
- **count():** count() -> int
 - Return the number of elements in this RDD.
 - rdd.count()
- **take(n):** take(n) -> list
 - Take the first n elements of the RDD.
 - rdd.take(5)
- **first():** first() -> T
 - Return the first element in this RDD.
 - rdd.first()
- **reduce(f):** reduce(f) -> T
 - Reduces the elements of this RDD using the specified commutative and associative binary operator.
 - rdd.reduce(lambda a, b: a + b)

RDD Actions (II)

- **saveAsTextFile(path):**
saveAsTextFile(path)
 - Save this RDD as a text file, using string representations of elements.
 - `rdd.saveAsTextFile("output")`
- **countByKey():** countByKey() -> dict
 - Count the number of elements for each key, and return the result to the master as a dictionary.
 - `rdd.countByKey()`
- **countByValue():** countByValue() -> dict
 - Return the count of each unique value in this RDD as a dictionary of (value, count) pairs.
 - `rdd.countByValue()`
- **max():** max() -> T
 - Return the max of this RDD.
 - `rdd.max()`
- **min():** min() -> T
 - Return the min of this RDD.
 - `rdd.min()`
- **sum():** sum() -> number
 - Add up the elements in this RDD.
 - `rdd.sum()`

PySpark SQL Theory

PySpark SQL is a Spark module for structured data processing. It provides a programming abstraction called **DataFrame**.

- DataFrames are distributed collections of data organized into named columns, similar to a table in a relational database.

- Spark SQL uses an optimizer called **Catalyst**, which makes DataFrame programs faster than RDD-based ones.

Class Descriptions

SparkSession: The entry point to programming Spark with the Dataset and DataFrame API. It is used to create **DataFrame**, register **DataFrame** as tables, execute SQL over tables, cache tables, and read parquet files.

DataFrame: A distributed collection of data organized into named columns. It is conceptually equivalent to a table in a relational database or a data frame in R/Python, but with richer optimizations under the hood.

Column: A column expression in a DataFrame. Column instances are created by selecting a column from a DataFrame.

GroupedData: Aggregation methods, returned by `DataFrame.groupBy()`. Used to perform aggregate operations on the grouped data.

SparkSession Functions

createDataFrame: createDataFrame(data, schema) -> DataFrame
Creates a DataFrame from an RDD, a list or a pandas.DataFrame.
`spark.createDataFrame([(1, 'a'), (2, 'b')], ['id', 'val'])`

read: read() -> DataFrameReader
Returns a DataFrameReader that can be used to read data in as a DataFrame.
`spark.read.format("csv").load("path")`

sql: sql(sqlQuery) -> DataFrame
Executes a SQL query and returns the result as a DataFrame.
`spark.sql("SELECT * FROM table")`

stop: stop() -> None
Stops the SparkSession.
`spark.stop()`

conf: conf() -> SparkConf
Returns the SparkConf object of this session.
`spark.conf.get("spark.sql.shuffle.partitions")`

range: range(start, end, step, numPartitions) -> DataFrame
Create a DataFrame with a single column named id, containing elements in a range.
`spark.range(1, 7, 2).show()`

udf: udf() -> UDFRegistration
Returns a UDFRegistration object for registering UDFs.
`spark.udf.register("my_udf", lambda x: x * 2)`

DataFrame Functions (I)

agg: agg(*exprs) -> DataFrame
Compute aggregates and returns a DataFrame.
`df.groupBy("age").agg({"*": "count"})`

alias: alias(alias) -> DataFrame
Returns a new DataFrame with an alias set.
`df.alias("my_df")`

cache: cache() -> DataFrame
Persists the DataFrame with the default storage level (MEMORY_AND_DISK).
`df.cache()`

coalesce: coalesce(numPartitions) -> DataFrame
Returns a new DataFrame with exactly numPartitions partitions.
`df.coalesce(1)`

collect: collect() -> list[Row]
Returns all the records as a list of Row.
`df.collect()`

columns: columns -> list[str]
Returns all column names as a list of strings.
`df.columns`

DataFrame Functions (II)

count: count() -> int

Returns the number of rows in the DataFrame.

df.count()

createOrReplaceTempView: cOTV(name) -> None

Creates or replaces a local temporary view with this DataFrame.

df.createOrReplaceTempView("people")

distinct: distinct() -> DataFrame

Returns a new DataFrame containing the distinct rows.

df.distinct()

drop: drop(*cols) -> DataFrame

Returns a new DataFrame that drops the specified columns.

df.drop('age', 'name')

crossJoin: crossJoin(other) -> DataFrame

Returns the cartesian product with another DataFrame.

df1.crossJoin(df2)

dropDuplicates: dropDuplicates(subset) -> DataFrame

Return a new DataFrame with duplicate rows removed, optionally only considering certain columns.

df.dropDuplicates(["name", "department"])

filter: filter(condition) -> DataFrame

Filters rows using the given condition.

df.filter(df['age'] > 21)

first: first() -> Row

Returns the first row as a Row.

df.first()

groupBy: groupBy(*cols) -> GroupedData

Groups the DataFrame using the specified columns.

df.groupBy("department")

join: join(other, on, how) -> DataFrame

Joins with another DataFrame.

df1.join(df2, df1.id == df2.id, 'outer')

limit: limit(n) -> DataFrame

Returns a new DataFrame by taking the first n rows.

df.limit(5)

orderBy: orderBy(*cols, ascending) -> DataFrame

Returns a new DataFrame sorted by the specified column(s).

df.orderBy("age", ascending=False)

persist: persist() -> DataFrame

Persists this DataFrame with the default storage level.

df.persist()

rdd: rdd -> RDD[Row]

Returns the content as an RDD of Row.

df.rdd

select: select(*cols) -> DataFrame

Projects a set of expressions and returns a new DataFrame.

df.select("name", "age")

show: show(n, truncate, vertical) -> None

Prints the first n rows to the console.

df.show(5)

sort: sort(*cols, ascending) -> DataFrame

Returns a new DataFrame sorted by the specified column(s).

df.sort(df.age.desc())

union: union(other) -> DataFrame

Return a new DataFrame containing union of rows from this and another frame.

df1.union(df2)

where: where(condition) -> DataFrame

An alias for filter().

df.where(df.age > 21)

withColumn: withColumn(colName, col) -> DataFrame

Returns a new DataFrame by adding a column or replacing an existing one.

df.withColumn("age_plus_1", df.age + 1)

withColumnRenamed: withColumnRenamed(old, new) -> DataFrame

Returns a new DataFrame by renaming an existing column.

df.withColumnRenamed("age", "user_age")

summary: summary(statistics) -> DataFrame

Computes specified statistics for numeric and string columns.

df.summary("count", "min", "max").show()

write: write() -> DataFrameWriter

Interface for saving the content of the non-streaming DataFrame out into external storage.

df.write.format("parquet").save("path")

Column Functions

alias: alias(alias) -> Column

Gives the column an alias.

df.age.alias("user_age")

asc: asc() -> Column

Returns a sort expression based on ascending order of the column.

df.sort(df.age.asc())

between: between(lower, upper) -> Column

A boolean expression that is evaluated to true if the value of this expression is between the given bounds.

df.select(df.name, df.age.between(18, 30))

cast: cast(dataType) -> Column

Casts the column to a different data type.

df.age.cast("string")

contains: contains(other) -> Column

A boolean expression that is evaluated to true if the value of this expression contains the given value.

df.filter(df.name.contains("o"))

desc: desc() -> Column

Returns a sort expression based on descending order of the column.

df.sort(df.age.desc())

Column Functions (II)

endswith: `endswith(other) -> Column`
String ends with another string.
`df.filter(df.name.endswith("n"))`

isNull: `isNull() -> Column`
True if the current expression is null.
`df.filter(df.age.isNull())`

isNotNull: `isNotNull() -> Column`
True if the current expression is NOT null.
`df.filter(df.age.isNotNull())`

like: `like(other) -> Column`
SQL like expression.
`df.filter(df.name.like("A%"))`

startswith: `startswith(other) -> Column`
String starts with another string.
`df.filter(df.name.startswith("A"))`

GroupedData Functions

agg: `agg(*exprs) -> DataFrame`
Compute aggregates and returns a DataFrame.
`df.groupBy("dep").agg({"salary": "avg"})`

avg: `avg(*cols) -> DataFrame`
Computes the average for numeric columns for each group.
`df.groupBy("dep").avg("salary")`

count: `count() -> DataFrame`
Counts the number of records for each group.
`df.groupBy("dep").count()`

max: `max(*cols) -> DataFrame`
Computes the max for numeric columns for each group.
`df.groupBy("dep").max("salary")`

sum: `sum(*cols) -> DataFrame`
Computes the sum for numeric columns for each group.
`df.groupBy("dep").sum("salary")`

mean: `mean(*cols) -> DataFrame`
Computes the mean for numeric columns for each group.
`df.groupBy("dep").mean("salary")`

min: `min(*cols) -> DataFrame`
Computes the min for numeric columns for each group.
`df.groupBy("dep").min("salary")`

pyspark.sql.functions (I)

abs: `abs(col) -> Column`
Computes the absolute value.
`from pyspark.sql.functions import abs`
`df.select(abs(df.value))`

avg: `avg(col) -> Column`
Returns the average of the values in a group.
`from pyspark.sql.functions import avg`
`df.select(avg(df.age))`

coalesce: `coalesce(*cols) -> Column`
Returns the first column that is not null.
`from pyspark.sql.functions import coalesce`
`df.select(coalesce(df.col1, df.col2))`

col: `col(col) -> Column`
Returns a Column based on the given column name.
`from pyspark.sql.functions import col`
`df.select(col("age"))`

concat: `concat(*cols) -> Column`
Concatenates multiple input string columns into a single string column.
`from pyspark.sql.functions import concat`
`df.select(concat(df.first, lit(' '), df.last))`

count: `count(col) -> Column`
Returns the number of items in a group.
`from pyspark.sql.functions import count`
`df.select(count(df.age))`

dayofmonth: `dayofmonth(col) -> Column`
Extract the day of the month as an integer from a given date/timestamp.
`from pyspark.sql.functions import dayofmonth`
`df.select(dayofmonth(df.date))`

lit: `lit(col) -> Column`
Creates a Column of literal value.
`from pyspark.sql.functions import lit`
`df.withColumn("one", lit(1))`

sum: `sum(col) -> Column`
Return the sum of the values in a group.
`from pyspark.sql.functions import sum`
`df.select(sum(df.salary))`

upper: `upper(col) -> Column`
Converts a string expression to upper case.
`from pyspark.sql.functions import upper`
`df.select(upper(df.name))`

when: `when(condition, value) -> Column`
Evaluates a list of conditions and returns one of multiple possible result expressions.
`from pyspark.sql.functions import when`
`df.select(when(df.age > 20, 1).otherwise(0))`

udf: `udf(f, returnType) -> UserDefinedFunction`
Creates a user-defined function (UDF).
`from pyspark.sql.functions import udf`
`from pyspark.sql.types import IntegerType`
`slen = udf(lambda s: len(s), IntegerType())`
`df.select(slen(df.name))`

aggregate: `aggregate(col, initialValue, merge, finish) -> Column`
Aggregates over a list of values.
`from pyspark.sql.functions import aggregate`

dense_rank: `dense_rank() -> Column`
Window function that returns the rank of rows within a window partition, without any gaps.
`from pyspark.sql.functions import dense_rank`

pyspark.sql.functions (II)

split: `split(str, pattern) -> Column`
Splits string `str` around occurrences that match `pattern`.

```
from pyspark.sql.functions import split
```

substring: `substring(str, pos, len) -> Column`
Substring starts at `pos` and is of length `len` when `str` is String type.

```
from pyspark.sql.functions import substring
```

trim: `trim(col) -> Column`
Trims the spaces from both ends for the specified string column.

```
from pyspark.sql.functions import trim
```

PySpark MLlib Theory

pyspark.ml is the newer, DataFrame-based API for MLlib. It is the recommended API.

- **Transformer:** An algorithm that can transform one DataFrame into another DataFrame. E.g., a feature transformer or a trained model. It has a `.transform()` method.
- **Estimator:** An algorithm which can be fit on a DataFrame to produce a Transformer. E.g., a learning algorithm like `LogisticRegression`. It has a `.fit()` method.
- **Pipeline:** Chains multiple Transformers and Estimators together to specify an ML workflow. The stages are run in order, and the input DataFrame is transformed as it passes through each stage.
- **Parameter:** All Transformers and Estimators share a common API for specifying parameters.

MLlib Feature Extraction (I)

VectorAssembler: `VectorAssembler(inputCols, outputCol) -> Transformer`

A feature transformer that merges multiple columns into a single vector column.

```
assembler = VectorAssembler(  
    inputCols=["c1", "c2"],  
    outputCol="features")
```

StandardScaler: `StandardScaler(inputCol, outputCol) -> Estimator`

An estimator that scales a vector column of features to have zero mean and unit variance.

```
scaler = StandardScaler(  
    inputCol="features",  
    outputCol="scaledFeatures")
```

StringIndexer: `StringIndexer(inputCol, outputCol) -> Estimator`

An estimator that encodes a string column of labels into a column of label indices.

```
indexer = StringIndexer(  
    inputCol="category",  
    outputCol="categoryIndex")
```

IndexToString: `IndexToString(inputCol, outputCol) -> Transformer`

A transformer that maps a column of indices back to a column of original string values.

```
converter = IndexToString(  
    inputCol="prediction",  
    outputCol="originalCategory")
```

Tokenizer: `Tokenizer(inputCol, outputCol) -> Transformer`

A transformer that converts the input string to lowercase and then splits it by white spaces.

```
tokenizer = Tokenizer(  
    inputCol="text", outputCol="words")
```

StopWordsRemover:

`StopWordsRemover(inputCol, outputCol) -> Transformer`

A transformer that filters out stop words from input.

```
remover = StopWordsRemover(  
    inputCol="words",  
    outputCol="filtered")
```

HashingTF: `HashingTF(inputCol, outputCol, numFeatures) -> Transformer`

A transformer that maps a sequence of terms to their term frequencies using the hashing trick.

```
hashingTF = HashingTF(  
    inputCol="filtered",  
    outputCol="rawFeatures")
```

IDF: `IDF(inputCol, outputCol) -> Estimator`
An estimator that computes the Inverse Document Frequency (IDF) and then scales the term frequencies.

```
idf = IDF(inputCol="rawFeatures",  
    outputCol="features")
```

MLlib Classification & Clustering

LogisticRegression: `LogisticRegression(featuresCol, labelCol) -> Estimator`

A classification algorithm for binary classification problems.

```
lr = LogisticRegression(  
    featuresCol='features',  
    labelCol='label')
```

DecisionTreeClassifier: `DTC(featuresCol, labelCol) -> Estimator`

A classification algorithm that predicts the class of a target variable by learning simple decision rules.

```
dt = DecisionTreeClassifier()
```

KMeans: `KMeans(featuresCol, k) -> Estimator`

A clustering algorithm that groups data points into a predefined number of clusters.

```
kmeans = KMeans(k=2)
```

MLlib Model Evaluation (I)

BinaryClassificationEvaluator:

`BCE(rawPredictionCol, labelCol) -> Evaluator`
Evaluator for binary classification, which expects two input columns: `rawPrediction` and `label`.

```
evaluator =  
    BinaryClassificationEvaluator()
```

MulticlassClassificationEvaluator:

`MCE(predictionCol, labelCol) -> Evaluator`
Evaluator for multiclass classification, which expects two input columns: `prediction` and `label`.

```
evaluator =  
    MulticlassClassificationEvaluator(  
        metricName="accuracy")
```

MLlib Model Evaluation (II)

evaluate: evaluate(dataset) -> float

Evaluates the model on a dataset and returns the metric.

accuracy = evaluator.evaluate(predictions)

MLlib Parameter Tuning

Pipeline: Pipeline(stages) -> Estimator

A pipeline chains multiple estimators and transformers together to specify an ML workflow.

pipeline = Pipeline(stages=[s1, s2, s3])

ParamGridBuilder: ParamGridBuilder() -> ParamGridBuilder

A builder for constructing a param grid used in grid search-based model selection.

grid = ParamGridBuilder().build()

CrossValidator: CrossValidator(estimator, eParamMaps, evaluator) -> Estimator

An estimator for k-fold cross validation.

cv = CrossValidator(estimator=pl,
estimatorParamMaps=grid,
evaluator=eval, numFolds=3)

Spark Streaming Theory

Spark Streaming processes a live input stream of data by dividing it into small batches.

- **DStream (Discretized Stream):** The basic abstraction. It is a sequence of RDDs, where each RDD contains data from a certain time interval.
- **Micro-batching:** Spark Streaming receives data and buffers it in memory. Then, the Spark engine runs a short task (a micro-batch) on the new data.
- **StreamingContext:** The main entry point for all streaming functionality.

Spark Streaming Context

StreamingContext: StreamingContext(sc, batchDuration) -> ssc

Create a new StreamingContext.

from pyspark.streaming import

StreamingContext

ssc = StreamingContext(sc, 10)

socketTextStream: socketTextStream(host, port) -> DStream

Creates an input stream from a TCP socket.

lines = ssc.socketTextStream("localhost",
9999)

start: start() -> None

Starts the execution of the streams.

ssc.start()

awaitTermination: awaitTermination() -> None

Waits for the execution to stop.

ssc.awaitTermination()

checkpoint: checkpoint(directory) -> None

Sets the checkpoint directory. Must be on a fault-tolerant file system. Required for stateful transformations.

ssc.checkpoint("hdfs:///checkpoints")

DStream Transformations

window: window(windowLength, slideInterval) -> DStream

Returns a new DStream which is computed based on windowed batches.

windowedStream = ds.window(30, 10)

reduceByKeyAndWindow: rBKW(func, invFunc, wL, sI) -> DStream

Aggregates values of each key over a sliding window. invFunc makes it more efficient by subtracting the values of the old window.

ds.reduceByKeyAndWindow(add, sub, 30, 10)

updateStateByKey: updateStateByKey(updateFunc) -> DStream

Allows you to maintain arbitrary state while continuously updating it with new information. It is applied on a DStream of '(key, value)' pairs.

ds.updateStateByKey(updateFunction)

transform: transform(func) -> DStream

Applies an RDD-to-RDD function to every RDD of the DStream. Allows for arbitrary RDD operations.

ds.transform(lambda rdd: rdd.sortByKey())

DStream Actions

pprint: pprint() -> None

Prints the first 10 elements of every batch of data in a DStream on the driver.

counts.pprint()

saveAsTextFiles: saveAsTextFiles(prefix, [suffix]) -> None

Saves the content of the DStream as text files. A new directory is created for each batch.

counts.saveAsTextFiles("output", "txt")

foreachRDD: foreachRDD(func) -> None

The most generic output operator. It applies a function to each RDD in the DStream.

ds.foreachRDD(lambda rdd:
rdd.toDF().write...)